

FlowDesk

Complete Technical Architecture & System Guide

A unified workspace connecting client CRM, projects, versioned deliverables, approvals, contracts, automated multi-currency invoicing, and Razorpay payment tracking.

Stack: Next.js 15 | React 19 | TypeScript | Tailwind CSS | Supabase RLS | Razorpay | Brevo

● **Full-Stack Architecture**

Next.js 15 App Router with TypeScript, server-side actions, route handlers, and modular frontend components.

● **Dual Auth & Security**

Production Supabase Auth with Row-Level Security (RLS) and standalone demo mode for testing.

● **Financial Engine**

Multi-currency invoice calculation, dynamic PDF generator, tax/discount logic, and Razorpay payment gateway.

● **Client Engagement Portal**

Secure client portal for interactive deliverable review, approval workflows, comments, and invoice checkout.

Document Metadata & Release Information

Attribute	Project Specification
Repository	https://github.com/ashutosh89-ctrl/Flowdesk_v1.git
Framework & Runtime	Next.js 15.4.9 (App Router) / Node.js 20+ / React 19.2.1
Styling & UI	Tailwind CSS v4, Lucide Icons, Framer Motion
Database Engine	PostgreSQL (Supabase) with Multi-Tenant Row Level Security
Payment Gateway	Razorpay Standard Checkout & Webhooks with Idempotency
Transactional Email	Brevo (Sendinblue) API & Resend Engine with HTML Templates
Document & PDF Engine	jsPDF, jspdf-autotable, html2canvas client/server renderer
AI Model Integration	Google Gemini API (@google/genai SDK v2.4.0)
Generated On	October 7, 2026

Section 1: Executive Summary & Product Vision

FlowDesk is a comprehensive, production-grade Freelancer Operating System designed to address the fragmentation inherent in modern solo business operations. Freelancers, digital agencies, and independent consultants traditionally rely on 5 to 7 disparate SaaS tools (CRM, cloud storage, document signing, deliverable proofing, invoice generation, payment gateways, and email alerts). FlowDesk unifies the entire client lifecycle into an interconnected, single-pane-of-glass workspace.

The End-to-End Client Value Loop

The application is architected around a strict hierarchical relational pipeline: Client -> Project -> Documents & Specifications -> Deliverable Submissions -> Interactive Approval / Revision Cycle -> Itemized Multi-Currency Invoice -> Online Payment Gateway Settlement & Audit Logging.

Stage	Functional Role & Capability
1. Client Management	Centralized client CRM recording contacts, active engagements, lifetime billing totals, status flags, and unique invitation tokens.
2. Project Scoping	Milestone planning, deliverable schedules, budgets, burn rates, deadlines, and real-time completion tracking.
3. Document Repository	Storage and requirement tracking for contracts, creative briefs, master service agreements, and client asset uploads.
4. Deliverables & Iterations	Upload files with semantic versioning (v1.0, v1.1, v2.0), status badges, internal notes, and one-click client review requests.
5. Interactive Client Portal	Dedicated client interface allowing stakeholders to inspect work, request revisions with timestamped annotations, or execute approvals.
6. Invoicing Engine	Itemized billing with custom tax percentages, discounts, dynamic number-to-words currency formatting, and instant PDF downloads.
7. Payment Settlement	Integrated Razorpay Checkout supporting Card, UPI, Netbanking, automatic status reconciliation, and immutable ledger recording.

Section 2: System Architecture & Technology Stack

FlowDesk utilizes the Next.js 15 App Router with full TypeScript strictness. The application is designed around clean layered separation between the presentation tier (frontend views and modals), the server orchestration tier (Next.js API routes and middleware), and the data storage tier (Supabase PostgreSQL / Browser LocalStorage).

Layer	Technology / Package	Architectural Role
Frontend Framework	Next.js 15.4.9 (App Router)	Server-side rendering, React Server Components, dynamic client-side interactivity, fast routing.
User Interface Library	React 19.2.1 + Tailwind CSS v4	Custom design system with CSS custom properties, glassmorphism, responsive grid layouts.
Icons & Animations	Lucide React + Motion (Framer)	Rich iconography and micro-interactions for polished user feedback and state transitions.
Database & Auth	Supabase (PostgreSQL 15+)	Row Level Security (RLS) multi-tenancy, UUID primary keys, relational foreign keys, cascade deletes.
Payment Gateway	Razorpay SDK v2.9.8	Order creation, SHA-256 HMAC signature verification, webhook reconciliation, test/live credentials.
Email Communications	Brevo API v6.26 & Resend	Transactional emails for client connection invitations, deliverable reviews, approvals, and invoice notices.
Document & PDF Engine	jsPDF & jspdf-autotable	Client-side and server-side PDF generation with custom headers, tables, tax breakdowns, and branding.
Form Validation	React Hook Form + Zod v4.4.3	Type-safe schema validation across user onboarding, client creation, deliverables, and invoices.
AI Assistance	Google Gemini (@google/genai)	AI-assisted generation of proposals, project scopes, brief analysis, and

Layer	Technology / Package	Architectural Role
		client communication.

Fail-Closed Dual-Mode Authentication Principle

FlowDesk supports two mutually exclusive modes controlled by NEXT_PUBLIC_AUTH_MODE: "production" enforces strict Supabase authentication where missing credentials fail closed without leaking demo data; "demo" activates an isolated, self-contained local storage environment for testing without external dependencies.

Section 3: Directory Structure & Codebase Breakdown

The repository follows a clean modular architecture separating backend logic, frontend modules, database migrations, and shared utilities:

Directory / Path	Description & Key Responsibilities
app/	Next.js App Router root containing page views, layouts, route handlers, and API endpoints.
-- api/auth/	Authentication callbacks, OAuth handling, session exchange, and post-login redirection.
-- api/payments/razorpay/	Endpoints for order creation, payment signature verification, and webhook callbacks.
-- api/invoices/	Invoice PDF generation, email dispatch, and settlement recording endpoints.
-- api/invitations/	Client portal connection links, token generation, validation, and acceptance.
-- dashboard/	Freelancer main management dashboard (KPIs, clients, projects, deliverables, invoices).
-- portal/[clientId]/	Client-facing portal view for viewing projects, deliverables, documents, and invoices.
-- connect/[token]/	Client onboarding & invitation acceptance view.
src/backend/	Domain business services, repository patterns, database access, and external integrations.
-- auth/	AuthService, ProfileService, SessionService, UserSettingsService, AccountDeletionService.
-- freelancer/	FreelancerClientService, FreelancerProjectService, DeliverableService, InvoiceService.
-- client/	ClientAuthService, ClientDeliverableService, ClientInvoiceService, ClientCommentService.
-- payments/	PaymentService, RazorpayClient, RazorpayService, currency utilities, subunit converters.
-- email/	BrevoClient, ResendClient, EmailService, responsive transactional HTML templates.
-- storage/	StorageHelper managing avatars, logos, signatures, documents, and deliverables buckets.
-- store/	FlowDeskStore (localStorage persistence engine for Demo Mode with mock seeding).
-- utilities/	Supabase client initializer, workspace resolver, notification helper, logger, rate limiter.
src/frontend/	React 19 UI views, modals, forms, and client components.
-- freelancer/	Views for Clients, Projects, Deliverables, Invoices, Documents, Settings, Activity.
-- client/	ClientPortalShell, DeliverablesPanel, InvoicesPanel, CommentsPanel, RazorpayCheckoutModal.
-- auth/	AuthContext, LoginForm, SignupForm, OnboardingFlow, ResetPasswordForm, VerifyEmail.
-- landing/	LandingPage, Hero, MonochromeFluidCanvas, Features, Pricing, Testimonials, FAQ.
src/shared/	Universal types, Zod schemas, currency utilities, date formatters, and PDF generators.
database/	Database schema and 20 chronological PostgreSQL migration files with RLS policies.
tests/	13 automated test suites verifying auth fail-closed, RLS red-teaming, payments, emails.
scripts/	Verification scripts for database hardening, Razorpay integration, RLS, and SEO/AEO.

Section 4: Database Schema & Relational Data Model

FlowDesk employs a multi-tenant PostgreSQL schema hosted on Supabase. Data isolation is strictly enforced through a hierarchy rooted in Profiles and Workspaces. Every core entity links to a workspace_id, preventing data leakage across freelancers.

Table Name	Columns & Keys	Description
profiles	id (UUID PK -> auth.users.id), full_name, business_name, email, avatar_url, phone, country, timezone, currency, onboarding_completed, created_at, updated_at	Stores user identity and profile metadata linked directly to Supabase Auth.
workspaces	id (UUID PK), owner_id (FK -> profiles.id), name, logo_url, created_at, updated_at	Top-level tenant boundary. All clients, projects, deliverables, and invoices belong to a workspace.
user_settings	id (UUID PK -> auth.users.id), currency, timezone, date_format, invoice_prefix, default_tax_rate, tax_name, email_notifications, invoice_reminders, comment_alerts	User preferences for formatting, default invoice numbering, tax rates, and alert toggles.
clients	id (UUID PK), workspace_id (FK -> workspaces.id), user_id (FK), name, company, email, phone, status, health_badge, active_projects_count, total_billed, country, currency, notes	Client CRM records. Represents external companies or individuals engaging the freelancer.
projects	id (UUID PK), workspace_id (FK), client_id (FK -> clients.id), title, description, status, budget, spent, completion_percentage, start_date, due_date, tags, milestones (JSONB)	Projects grouped under clients with financial budgeting and milestone tracking.
deliverables	id (UUID PK), workspace_id (FK), client_id (FK), project_id (FK), title, description, status, due_date, review_requested_at, approved_at, rejection_reason, current_version, priority	Individual work assets submitted for client review (Draft, In Review, Needs Changes, Approved).
deliverable_versions	id (UUID PK), deliverable_id (FK -> deliverables.id), version_number, file_url, file_name, file_size, note, uploaded_by, archived, created_at	Immutable history of deliverable iterations (e.g., v1.0, v1.1, v2.0) with attached files.
deliverable_comments	id (UUID PK), deliverable_id (FK), author, author_id, author_role, is_internal, content, attachments, reply_to_id, resolved, created_at	Threaded feedback between freelancer and client with internal-only visibility flags.
documents	id (UUID PK), workspace_id (FK), client_id (FK), title, type, status, is_required, due_date, description, file_name, file_url, version	Contracts, creative briefs, specifications, and client document upload requests.
invoices	id (UUID PK), workspace_id (FK), client_id (FK), project_id (FK), invoice_number, status, issue_date, due_date, subtotal, tax_percentage, tax_amount, total_amount, paid_amount, currency	Financial invoices tracking itemized billing, taxes, outstanding balances, and payment status.
invoice_items	id (UUID PK), invoice_id (FK -> invoices.id), description, quantity, unit_price, amount, created_at	Line items associated with each invoice.
invoice_payments	id (UUID PK), invoice_id (FK -> invoices.id), amount, payment_method, razorpay_order_id, razorpay_payment_id, razorpay_signature, gateway, gateway_status, currency, captured_at	Audit ledger of all recorded payments (manual bank transfers, cash, or Razorpay online payments).
razorpay_orders	id (UUID PK), order_id (TEXT UNIQUE), invoice_id (FK), workspace_id (FK), client_id (FK), amount, amount_subunits, currency, status, receipt, notes (JSONB)	Payment intents generated for Razorpay checkout tracking order lifecycle and webhook reconciliation.
activities	id (UUID PK), workspace_id (FK), user_id (FK), client_id (FK), project_id (FK), action, title, description, user_name, resource_type, resource_id, created_at	Chronological audit trail of all actions across the workspace.
notifications	id (UUID PK), workspace_id (FK), user_id (FK), title, message, category, link, read, priority, created_at	In-app notification records for approvals, feedback, overdue invoices, and system updates.

Section 5: Authentication, Authorization & Security Architecture

FlowDesk implements an enterprise-grade security model preventing unauthorized access, cross-tenant data leakage, and silent fallback vulnerabilities. The security model is split across four protective layers:

Security Layer	Implementation Details
1. Route Middleware Protection	The Next.js edge middleware (middleware.ts) intercepts all incoming requests. Protected routes (/dashboard,

Security Layer	Implementation Details
	/portal/[clientId], /client/*, /api/*) verify Supabase authentication cookies. Unauthenticated requests are immediately redirected to /login with return URLs.
2. Multi-Tenant Row Level Security	PostgreSQL RLS policies are enabled on 100% of tables. Freelancers can strictly read/write records where workspace.owner_id = auth.uid(). Clients are granted scoped SELECT and UPDATE permissions only for rows matching their assigned client_id.
3. Fail-Closed Authentication Strategy	Phase 28A security audit eliminated silent demo fallbacks. If Supabase is unreachable or misconfigured in production mode, the application surfaces an explicit configuration error rather than loading mock data.
4. Credential & Secret Isolation	Sensitive API keys (SUPABASE_SERVICE_ROLE_KEY, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET, BREVO_API_KEY) are restricted exclusively to Node.js server runtimes and never leaked to client bundles.

Authentication Flows

- Freelancer Auth: Supports standard Email/Password authentication, OAuth (Google and GitHub), Magic Link, and Password Reset. Upon first signup, the user is routed through a 3-step onboarding flow to initialize business profile details and default workspace settings.
- Client Portal Access: Clients receive cryptographically secure invitation links (/connect/[token]) generated by the freelancer. Upon acceptance, client records are mapped to auth identities, enabling single-click access to /portal/[clientId].

Section 6: Core Modules & Step-by-Step Workings

Module 1: Dashboard & Workspace Management

The Freelancer Dashboard provides real-time financial and operational intelligence. It aggregates total monthly revenue, pending invoice receivables, active project counts, and deliverables awaiting review. An interactive activity stream records real-time updates across all clients.

Module 2: Client CRM & Invitation System

Freelancers manage client directories with full contact information, currency preferences, tax IDs, and lifetime billing statistics. Freelancers can generate expiring invitation links via the Client Connection Service to onboard clients to their private portal.

Module 3: Project & Milestone Scoping

Projects feature customizable milestones, budget allocations, start/end dates, and progress percentage bars. Milestones automatically sync with deliverables, giving both freelancer and client full visibility into milestone completion rates.

Module 4: Deliverable Versioning & Approval Pipeline

Deliverables follow a structured four-stage Kanban lifecycle: Draft -> In Review -> Needs Changes -> Approved. Each deliverable supports multiple version uploads (v1.0, v1.1, etc.) stored securely in Supabase Storage with threaded comments and status flags.

Module 5: Document Library & Asset Request Engine

Manages legal and creative assets including Master Services Agreements (MSAs), Non-Disclosure Agreements (NDAs), project briefs, and brand assets. Includes status tracking (Pending, Uploaded, Approved) and file download permissions.

Module 6: Multi-Currency Invoicing & PDF Generation

Dynamic invoice builder with itemized entries, quantities, unit rates, tax percentages, and discount calculations. Includes an automated number-to-words currency generator and client-ready PDF generation via jsPDF with auto-table formatting and custom branding.

Module 7: Razorpay Payment Integration & Reconciliation

Seamless payment flow: Client opens invoice -> Clicks "Pay Online" -> Backend creates Razorpay order -> Razorpay Checkout modal appears -> Client completes payment via UPI/Card/Netbanking -> Backend verifies HMAC signature -> Invoice is automatically marked as Paid -> Email receipt is dispatched.

Module 8: Client Engagement Portal

Authenticated, isolated client interface allowing clients to view ongoing project timelines, approve or request revisions on deliverables, download contracts, and settle outstanding invoices directly.

Module 9: Transactional Email Notification Engine

Integrated with Brevo and Resend. Automatically triggers responsive HTML emails when client invitations are created, deliverables are submitted for review, revisions are requested, and invoices are issued or paid.

Section 7: Razorpay Payment Gateway Architecture

FlowDesk implements a robust, idempotent payment architecture integrating Razorpay Standard Checkout and Server-side Webhooks:

Payment Stage	Technical Implementation & Security
1. Order Creation	POST /api/payments/razorpay/create-order receives the invoice ID, validates the outstanding balance, converts the amount into minor currency units (e.g. paise/cents), and registers a Razorpay Order ID stored in the razorpay_orders table.
2. Client Checkout	The frontend mounts the Razorpay SDK checkout modal displaying the freelancer business branding, accepted payment methods (UPI, Cards, Netbanking, Wallets), and currency denomination.
3. Signature Verification	Upon successful payment, the client SDK returns razorpay_payment_id, razorpay_order_id, and razorpay_signature. The backend verifies the HMAC SHA-256 signature using the server secret key before recording the transaction.
4. Idempotent Settlement	PaymentService checks for unique payment IDs before inserting records into invoice_payments, incrementing paid_amount, and updating invoice status to "paid" or "partially_paid".
5. Webhook Redundancy	POST /api/payments/razorpay/webhook listens for payment.captured events to reconcile transactions in real-time even if the client closes the browser before redirection.

Section 8: Testing Suite & Quality Verification

FlowDesk includes 13 automated test suites and verification scripts ensuring strict data integrity, security compliance, and error resistance:

Test Suite / Script	Test Scope & Assertions
tests/auth-fail-closed.test.ts	Verifies that production authentication fails closed without leaking mock data when database credentials are unset.
tests/rls-redteam.test.ts	Executes red-team multi-tenant attack simulations verifying that User A cannot read or write User B workspaces, clients, or invoices.
tests/razorpay-payment.test.ts	Validates order creation, signature hashing, concurrency handling, and partial payment reconciliation.
tests/brevo-email-deep-audit.test.ts	Tests transactional email rendering, template parameters, error retries, and API dispatch.
tests/data-workflow-e2e.test.ts	Full end-to-end simulation: Client Creation -> Project -> Deliverable -> Approval -> Invoice -> Payment.
tests/rate-limiter.test.ts	Verifies IP-based and user-based token bucket rate limiting on public API endpoints to prevent abuse.

Section 9: Configuration & Deployment Guide

FlowDesk is optimized for zero-downtime deployment on Vercel, Supabase, and Cloud Run. Follow these environment configurations:

Environment Variable	Purpose & Environment
NEXT_PUBLIC_APP_URL	Production domain URL (e.g., https://flowdesk.app) used for links and OAuth callbacks.
NEXT_PUBLIC_AUTH_MODE	Set to "production" for real Supabase authentication or "demo" for isolated local testing.
NEXT_PUBLIC_SUPABASE_URL	Supabase PostgreSQL project API endpoint URL.
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase anonymous public API key for client-side authentication and RLS queries.
SUPABASE_SERVICE_ROLE_KEY	Supabase service role key (Server-side only) for administrative tasks.
RAZORPAY_KEY_ID / SECRET	Razorpay API Key ID and Secret for payment order generation and webhook verification.
NEXT_PUBLIC_RAZORPAY_KEY_ID	Public Razorpay Key ID used by the client-side checkout SDK modal.
BREVO_API_KEY / EMAIL_FROM	Brevo API key and verified sender address for sending transactional emails.

Deployment Command Quick Reference

- To build and start the production instance:
- 1. npm install (Install dependencies)
 - 2. npm run typecheck (Validate TypeScript integrity)
 - 3. npm run build (Generate optimized Next.js production bundle)
 - 4. npm start (Launch production server on port 3000)